2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C) | 978-1-6654-7836-6/21/$31.00 ©2021 IEEE | DOI: 10.1109/QRS-C55045.2021.00163

Let's Supercharge the Workflows: An Empirical Study of GitHub Actions

Tingting Chen, Yang Zhang*, Shu Chen, Tao Wang, Yiwen Wu
National University of Defense Technology, Changsha, China
{chentingting20, yangzhang15, chenshu16, wangtao2005, wuyiwen14}@nudt.edu.cn

***Abstract*—Automation has become a norm in software development practices, especially in CI/CD practices. Recently, GitHub introduced GitHub Actions (GA) to provide automated workflows for software maintainers. However, few researches have been proposed to evaluate its capability and impact, even though several GA have been built by practitioners. In this paper, we conduct a large-scale empirical study of GitHub projects, to help practitioners gain deep insights into the GA. We quantitatively investigate the basic adoption of GA and its potential correlation with project properties. We also analyze the usage details of GA, including its component scale and action sequences. Finally, using regression modeling, we investigate the impact of GA on the commit frequency, pull request and issue's resolution efficiency. Our findings suggest a nuanced picture of how practitioners are adapting to, and benefiting from GA.**

***Index Terms*—DevOps, CI/CD, GitHub Actions**

I. Introduction

Today, source code is essentially required to have Continuous Integration and Continuous Delivery/Deployment (CI/CD) to target environments because automation has become a norm in software development practices [5]. To carry out CI/CD, GitHub, as one of the most widely used source code repository providers, has integrated many third-party CI/CD tools, such as Travis CI [34], Jenkins [1] and Docker [28]. It is becoming increasingly clear that making a choice of CI/CD tools and infrastructures is a tough task [21]. To facilitate a state-of-the-art CI/CD workflow platform inside, GitHub recently introduced GitHub Actions[1] (GA), which allows developers to automate, customize, and execute the software development workflows within GitHub. This emerging feature makes it easier to automate build, test, and deploy software projects [5], following the notion of DevOps.

Recently, many researchers have investigated how CI/CD technologies affect project development productivity and efficiency, e.g., Travis CI [34], GitLab Linter [24], and Docker [27], [33]. Their research has yielded many interesting findings and brought many practical implications to developers to help them better work with CI/CD. GA provides a higher level, more integrated CI/CD environment which is different from those tools that only focus on a single CI/CD stage. In GA service, developers can create *actions* or utilize existing *actions* and create or customize *workflows* to perform any *job* or automate software development life cycle processes. However, we know relatively little about the state of the practice in using this technology and whether there is a positive impact of GA aligned with the GitHub officially claimed. Such knowledge can assist developers to optimize their CI/CD practices, maintainers to make informed decisions about adopting GA, and researchers and service/tool builders to find areas that need attention.

In this paper, we report on an empirical study of a large sample of GitHub projects that adopted GA. Specifically, we first analyze the basic adoption information of GA, i.e., investigating how many popular projects adopt GA in their development process and how presence / absence of the GA correlates with project properties. Then, we investigate the usage details of GA including its component scale and action sequences. Finally, we focus on the transition to using GA, and investigate how development practices changed following this shift. To this end, we use *Regression Discontinuity Design (RDD)* to quantitatively evaluate the effect of an intervention, in our case adoption of GA, on the commit frequency, pull request (PR) and issue resolution's efficiency. Finally, we distill many practical implications for different stakeholders.

In summary, the highlights of our findings are:

- Nearly 22% of popular projects in our dataset adopted GA. And projects that have larger contributors size tend to adopt GA.
- The average number of configured workflows per project is 2.8. Most jobs use servers hosted by GitHub and Ubuntu is the most used platform. Moreover, the steps in the GitHub Marketplace are most commonly used.
- In those Java projects in our dataset, two configuration rules among two steps can be found, but the occurrences of further rules are rare.
- Commit frequency is decreasing after switching to GA, but it is only accompanied by a small change.
- While the number of closed issues increases, this trend reverses by GA. And the issue solution latency decreases despite the commit frequency becoming slower.
- After initial adjustments when adopting GA, the number of pull requests seems to decrease. And the PR resolution latency tends to decrease after the adoption of GA.

Paper organization: Sec.II introduces the preliminaries. Sec.III elaborates study methodology. Sec.IV and Sec.V present study results and discussion, respectively. Sec.VI introduces the related works and Sec.VII concludes this paper.

*Yang Zhang is the corresponding author.

[1]https://github.com/features/actions. The feature was released to the public in November 2019.

2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C)

2693-9371/21/$31.00 ©2021 IEEE
DOI 10.1109/QRS-C55045.2021.00163

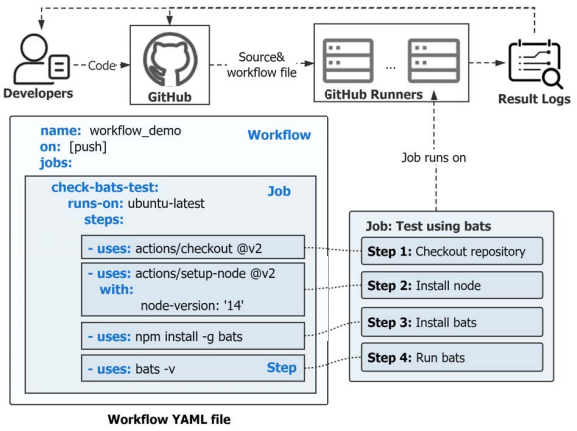

Fig. 1. An example of GitHub Actions workflow.

II. Preliminaries

A. Overview of GA

GitHub Actions (GA,) are a set of actions in a GitHub repository workflow, which allow developers to customize and execute software development workflows. Essentially, it is an event-driven API provided by GitHub to automate tasks within the software development life cycle. Developers can run a series of commands after a specified event (e.g. commits, pull requests, issues, etc) has occurred. There are several key concepts in GA. Specifically, a *workflow* is made up of one or more *jobs* and can be scheduled or triggered by an event. A *workflow* is configured in a YAML file that can be stored in the GitHub repository. Moreover, the files generated when developers build or test the software project are *artifacts* [4]. They can be created in one job and used in another for deployment in a workflow. Once a code change is pushed, or a pull request is made, an *event* can be set up in GA to trigger the workflow. Developers can configure external triggers using a repository dispatch webhook or use many other webhooks, e.g., deployment, workflow dispatch, and check runs. A *job* is a set of steps that execute on the same runner. A task that is an action or a command is identified as a *step*. The file system's information is shared with multiple steps (actions and commands) in a single job.

Fig.1 shows an example of GA workflow. There is one job for test in this workflow, which has a set of steps that execute on the same runner. There are four steps in this job, *'actions/checkout@v2'* and *'actions/setup-node@v2'* are the community actions. *'npm install -g bats'* and *'bats -v'* are the commands running on the runner. The first action checks out the repository and downloads it to the runner, allowing to run steps against the code. The second installs a specified version of the *node* software package on the runner, which gives developers access to the *npm* command. The third uses *npm* to install the *bats* software testing package. The last runs the *bats* command with a parameter that outputs the software version. Then developers can see whether the result is successful, failed, or canceled, if the run is completed. If the run is failed, they can get the build logs to diagnose the failure and re-run the workflow.

B. Research Questions

This study seeks to answer three research questions:

RQ1: (Basic adoption) *- What is the percentage of popular projects that adopt GA?*

In the first RQ, we aim to understand how commonly projects adopt GA, especially those popular projects (with 1,000+ stars). Because popular projects tend to receive more code contribution and development tasks, all of which need to be tested and integrated before deployment. One of the most commonly advocated benefits of GA is the ability to "scale up" development, by automating testing and delegating the workload to cloud-based infrastructure. Kinsman et al. [16] found that only 0.7% of active projects adopt GA. However, they didn't distinguish those popular projects. Whether popular projects are more likely to adopt GA needs to be examined. Part of our study can be considered an extended replication of Kinsman et al.'s [16] study: we have similar research goals, but we focus more on those popular projects and use statistical techniques to find whether certain types of projects tend to adopt GA more than others, i.e., how the presence/absence of the GA correlates with the properties of a project.

RQ2: (Usage details) *- How do developers configure GA?*

In the second RQ, our goal is to investigate how developers configure their GA in practice and if there are some common specifications on GA configuration. GA services allow developers to customize workflows, which define the sequential steps of jobs that are triggered by changes to the project. All the job scripts are expressed in the GA specification (i.e., YAML files). However, little is known about the details of GA specifications. Given that GA is an emerging feature, a possible reason for the non-adoption of GA is due to lacking knowledge about it. It seems reasonable that a GA template generation framework may be useful to ease the burden of GA adoption. As a step towards such a template generation framework, we want to perform an analysis of how many workflows, jobs, and steps, and which commands, are being used in real-world GA specifications.

RQ3: (Adoption effects) *- What is the impact of GA?*

In the last RQ, we seek to examine whether transitioning to GA affects ordinary development practices. Zhao et al. [34] found that after adopting CI, developers commit code more frequently. Similar to CI, GA may help developers focus more on coding work, thereby producing commits more frequently. As mentioned before, although popular projects tend to have more PRs, GA may automate the code testing and facilitate the resolution of PRs. For GA, and DevOps in general, to have the stated benefits, effective coordination between all project contributors is important; a common mechanism for this is issue report. Similar to PRs, the number of closed issues per time period may change after GA adoption, in response to increased project activity or efficiency. Moreover, this high level of GA's automation should also impact the speed, not just the volume. Thus, we want to examine how GA adoption

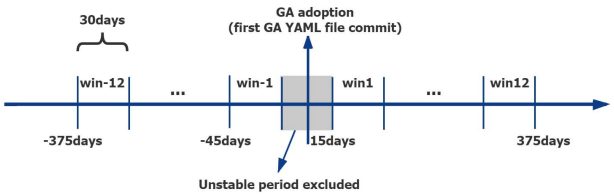

Fig. 2. Overview of our time series assembly method.

affects the PR and issue's resolution, including the size of closed items and the resolution latency.

III. Methodology

A. Data Collection

We filter and collect our data from the population of open-source projects on GitHub by using the GitHub API v3[2].

1) Projects Selection: The goal is to select nontrivial projects that use GA. We start by sorting GitHub projects (repositories) by their popularity (via the number of stars [10]). In this work, we only consider those popular projects with more than 1,000 stars. All of them are active, large, non-forked, and non-duplicated. This yields 27,790 projects, as of February 1, 2021. Next, we identify whether a project adopts GA by checking whether it has files of YAML format in the "*./github/workflows*" directory. We write a script to iterate over all candidate projects and identify 6,246 projects.

2) GA Data Gathering: We clone all the 6,246 projects locally and extract their YAML files. Then, we iterate over the project list and parse the YAML files kept in the "*.github/workflows*" directory. The *bashlex* parser library of Python is used to get the actions data as it creates an abstract syntax tree that maintains the inherent structure of jobs and the steps under them. We parse each line word by word to extract the *job name*, *step name*, *action name* and the orders in each job. To get a refined list of actions, we apply various regular expressions to avoid getting irrelevant data for the study such as *action version* and *conditional statements*. Notice that we focus on the actions instead of shell commands, so the steps which contain shell commands (including shell commands more than one line) are ignored. Finally, we obtain 47,431 actions and their detailed configuration data.

B. Quantitative Analyses

1) Time Series Analysis: To determine the impact of GA on development practices change, we conduct a time series analysis that uses longitudinal data in our dataset. We aggregate data about the different practices considered in 30-day windows, 12 on each side around the GA adoption event, as shown in Fig. 2. Similar to previous work [25], [34], we exclude one month of data centered around the adoption event in our quantitative analyses below to limit the risk of bias due to extraordinary project activity in the transition period. Since many projects on GitHub are inactive [8], we filter out projects inconsistently active during our 24-month observation period to avoid biasing our conclusions due to an inflation of zero values in our data.

[2]https://developer.github.com/v3/

2) Measure Factors: We collect global and time-window measures:

- **total commits** (totalCommits) in a project's history, as a proxy for project activity.
- **number of contributors** (contributor), as a proxy for the size of a project's community [32]; contributors include both core developers and external contributors.
- **age at GA** (ageAtGA), the time from the earliest commit of GA YAML file to February 1, 2021, in months.
- **main programming language** (language), automatically computed by GitHub based on the contents of each repository and extensions of file names therein.
- **number of non-merge commits, number of merge commits** (non-mergeCommits, mergeCommits) per time window [34]. We recognize non-merge commits as those having at most one parent, and merge commits otherwise.
- **number of issues closed, number of PRs closed** per time window, extracted using the GitHub API.
- **mean issue latency**, **mean PR latency** per time window, in hours, computed as the difference between the timestamp when the issue / PR was closed and that when it was opened. The mean is computed over all issues / PRs in that time window.

3) Regression Modeling: We use statistical modeling to discover longitudinal patterns indicative of GA adoption effects. We model the effect of GA adoption over time across GitHub projects using Regression Discontinuity Design (RDD) [22]. RDD is a technique used to model the extent of a discontinuity at the moment of intervention and long after the intervention. The technique is based on the assumption that in the absence of the intervention, the trend of the function would be continuous in the same way as prior to the intervention [13]. Let y be the outcome variable in which we are looking for a discontinuity, e.g., the number of merge commits per month. We specify this linear regression model to estimate the trend in y before GA adoption, and the changes in trend after adoption. The statistical model behind RDD is (1):

$$y_i = \alpha + \beta \cdot time_i + \gamma \cdot intervention_i + \delta \cdot time\_after\_intervention_i + \eta \cdot controls_i + \varepsilon_i \quad (1)$$

To model the passage of time as well as the GA adoption, we rely on three variables: *time*, *time_after_intervention*, and *intervention*. The *time* variable indicates the time in months at time *i* from the start of the observation period. We consider a time period of 24 months for this study, 12 months before and after GA adoption. The *intervention* variable is a binary value to indicate whether the time *i* occurs before (=0) or after the (=1) adoption event. The *time_after_intervention* variable counts the number of months at time *i* since the adoption event, the variable is set to 0 before adoption and $(time - 12)$ after adoption. The controls i variables enable the analysis of GA adoption effects, rather than confounding the effects that influence the dependent variables. For observations before the intervention, holding controls constant, the resulting regression line has a slope of β, and after the intervention $\beta + \delta$. The size

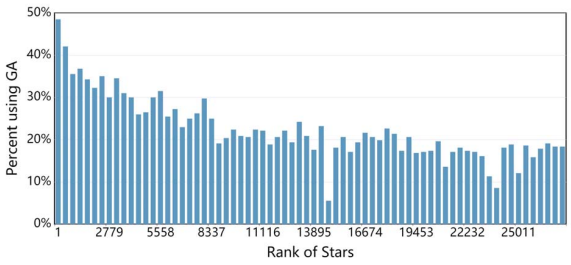

Fig. 3. GA adoption rate in projects with different popularity.

of the intervention effect is measured as the difference equal to γ between the two regression values of y_i at the moment of the intervention.

Following the work of Wessel et al. [26], we model project name and programming language as random effects to capture project-to-project and language-to-language variability. We evaluate the model fit using marginal (R^2_m) and conditional (R^2_c) scores [19]. The R^2_m can be interpreted as the variance explained by the fixed effects alone, and R^2_c as the variance explained by the fixed and random effects together. We report the significant coefficients in the regression as well as their variance, which are obtained by using ANOVA. In addition, we log transform the fixed effects and dependent variables that have high variance [14]. We also account for multicollinearity, excluding any fixed effects for which the variance inflation factor (*VIF*) is higher than 8 [14].

IV. Study Results

A. RQ1: Basic Adoption of GA

1) What is the percentage of popular projects that adopt GA?: First, we seek to investigate the adoption of GA in those popular projects (with 1,000+ stars). We find that 22% of projects adopt GA in their development history. This ratio is much higher than the finding of Kinsman et al. [16] (0.7% of the 416,266 repositories). Considering that there is a large proportion of unpopular projects in GitHub, it indicates that popular projects tend to adopt GA than unpopular projects. Our intuition is that GA may lead to better outcomes, i.e., there is a higher utilization rate of GA among the more popular projects (alternatively, those projects using GA tend to be more popular). To verify that, we divide the projects into 70 groups (397 projects per group) which are ordered by the number of stars. Then we calculate the percentage of projects in each group that adopt GA. Fig. 3 shows that the more popular projects are more likely to use GA. We find that the proportion of adopting GA gradually declines with the decline in popularity. 48% of projects use GA in the most popular group. As the projects become less popular, the percentage of projects using GA declines to 18%.

Finding 1 ☛ *Nearly 22% of projects in our dataset adopt GA in their development history. Moreover, more popular projects tend to have a higher proportion of GA adoption.*

2) Do certain types of projects adopt GA more than others?: Next, we want to check how the presence/absence of the GA correlates with the properties of a project, i.e., programming language, project age, and contributors size.

TABLE I
GA Adoption among Projects with Different Languages.

Language	Number of Projects	Number of Projects with GA	Percentage
TypeScript	1,118	588	52.59%
PHP	1,042	390	37.43%
Go	1,706	614	35.99%
C#	696	213	30.60%
C++	1,335	395	29.59%
Ruby	785	213	27.13%
Shell	620	145	23.39%
C	1,077	245	22.75%
Java	2,613	567	21.70%
Python	3,390	734	21.65%
JavaScript	5,516	937	16.99%
Swift	764	109	14.27%
HTML	757	81	10.70%
CSS	448	27	6.03%
Objective-C	871	41	4.71%

Programming language: We rank the number of projects by the percentage that adopt GA in each language and select the Top-15 programming languages. Table I shows the number of projects from our dataset that predominantly use some language, how many of these projects adopt GA, and the percentage of projects that adopt GA. We find that, the languages that most adopt GA are TypeScript, PHP, and Go. While among projects that adopt GA less, there are some popular languages such as JavaScript and Swift, as well as less popular languages such as CSS. We also observe that many of the languages that have high GA usage are also statically-typed languages (e.g., TypeScript, Go, C#, and C++). Programming languages may correlate with the adoption of GA, so we consider it as a random intercept in our models.

Project age: Next, we investigate the relationship between GA adoption and project age. For each project, we compute its age as the months that the project was created on GitHub to February 2021. Fig. 4 left shows the age distribution of projects with GA and without. For projects without GA, their average age is 68 months (median: 66 months), while projects with GA have an average age of 65 months (median: 64 months). Using the Wilcoxon test [7], we have established that the difference between the two groups is statistically significant (p<.0001), but their effect size is very small (Cliff's delta: 0.05)[3]. Therefore, we find that there is a weak correlation between GA adoption and project age.

Contributors size: We try to figure out whether projects with more contributors are more likely to adopt GA. Fig. 4 right shows the distribution of contributors size for projects with GA and without. We find that the projects without GA have an average of 34 contributors (median: 14), while projects with GA have an average of 96 contributors (median: 56). The difference between the two groups is statistically significant (p<.0001). Their effect size (Cliff's delta: 0.54) shows that the adoption of GA is correlated with the contributor size. One possible explanation may be that large-team projects have

[3]The magnitude is assessed using the thresholds, i.e., $|\delta|$<.147 "negligible", $|\delta|$<.33 "small", $|\delta|$<.474 "medium", otherwise "large".

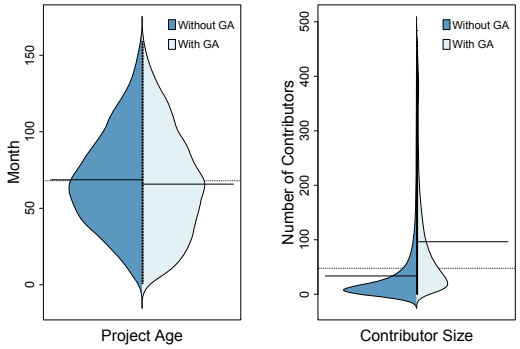

Fig. 4. Distributions of age and contributors size of projects w./wo. GA. Horizontal lines depict means.

more tasks [15] and adopting GA would help them reduce the development burden. Besides, developers may adopt GA to optimize their development workflows as the projects with more contributors are more likely well-maintained [29]. We consider it as a fixed intercept in the regression modeling.

Finding 2 ☛ *Programming languages may have an influence on GA adoption. And projects with a large contributors size tend to adopt GA. However, project age affects the adoption of GA weakly.*

B. RQ2: Usage Details of GA

1) How many components in a GA specification?: We extract YAML files under the workflow path in the projects that adopt GA. Each YAML file is configured with a workflow, then we extract the job in each workflow and the step in each job. Below are detailed statistics.

Workflow: We first analyze the workflow, which is an automated procedure made up of one or more jobs and can be scheduled or triggered by an event. The workflows are usually used to build, test, package, release, or deploy a project on GitHub. We find that, on average, each project is configured with 2.8 workflows (median: 2) and the maximum is 30 workflows. Through manual inspection, we find that the project with 30 workflows is configured with workflows for each operating environment. 99.14% of the workflows' status are *active*, 0.86% of the workflows' status are *manually disabled*. Excessive workflows may be caused by using GA incorrectly or requiring specific responses to different events.

Job: Next, we analyze the job, which is a set of steps executed on the same runner. We find that there are 1.8 jobs on average in each workflow (median: 1), and the maximum is 72 jobs. Each job in the workflow has an independent runner. These runners can be servers hosted by GitHub or self-hosted servers. GitHub-hosted runners are based on Ubuntu Linux, Microsoft Windows, and macOS. Each job with a GitHub-hosted runner runs in a fresh virtual environment. In addition, they can host their own runners if developers need other operating systems or require a specific hardware configuration. We extract the '*runs_on*' of each job, which represents the device on which the job runs. We calculate the distribution of jobs' hosts. As Table II shown, most jobs use servers hosted by GitHub (88.26%), of which Ubuntu is the most used and Windows is the least; the number of jobs using self-hosted servers is the least. Further, we extract the name of each job named by developers and get the following categories by using regular expressions and manual classification. Table III shows the type of job name which counts more than 100. We find that the job name in the workflow has two basic categories, the first category is named for the purpose which includes build, test, release, check, lint, etc. The second is named by the development language or platform, e.g., PHP, Windows.

TABLE II
DISTRBUTION OF THE HOST OF JOBS.

Runs_On		Number of Jobs	Percentage
Self-host		137	0.75%
GitHub host	Ubuntu	14,013	76.94%
	Windows	817	4.49%
	macOS	1,245	6.84%
Matrix		2,001	10.99%

TABLE III
DISTRIBUTION OF THE TYPE OF JOB NAMES.

Type of Jobname	Number
Build	4,508
Others	4,265
Test	3,305
Release	737
Check	670
Lint	639
Linux	439
Publish	422
Deploy	391
Windows	380
Analyse	327
Mac	263
CI	215
Doc	193
Docker	149
PHP	133
Format	132

TABLE IV
DISTRIBUTION OF THE TYPE OF STEPS.

Type of Steps	Number	Percentage
Public repository (GitHub Marketplace)	35,635	98.48%
Created by developers themselves	538	1.13%
Docker image	184	0.39%

Step: Finally, we analyze the step. Each step is a single task that can be run in a job, which can be an action or a shell command. The steps of one job run on the same runner and share data. Notice that our study only considers those steps that are standalone actions. Steps are the smallest portable building blocks of a workflow. Developers can create their own steps, or use steps created by the GitHub community. We extract and analyze the steps used in the job. There are a total of 47,431 steps, each job has 4.7 steps on average (median: 3). These steps are mainly divided into three categories: the steps (public repository) in the GitHub Marketplace, steps created by developers themselves, and steps using the Docker image. Table IV shows that the steps (public repository) in the GitHub

Marketplace account for the majority (98.48%), while the steps in their own repository and the steps using Docker image is rarely used. It can be observed that the step used by most projects is the first type, and the step has been reused widely.

Finding 3 ☛ *During GA configuration, the average number of configured workflows per project is 2.8, workflows have an average of 1.8 jobs, and jobs have an average of 4.7 steps. Only a small set (0.86%) of workflows are manually disabled by developers. In the workflows, most jobs use servers hosted by GitHub, additionally, Ubuntu is the most used platform. In addition, most names of jobs have two basic categories, including the usage purpose and the development language / platform. There are 3 types of steps and the steps in the public repository (GitHub Marketplace) are most commonly used by developers.*

2) Is there any specification on popular action sequences of GA?: We analyze and validate the popular action sequences in GA configuration using Sequential PAttern Discovery Equivalence classes (SPADE) [30]. In our analysis, we find projects with different languages have different environments which leads to different patterns. As a preliminary attempt, we focus our sequence analysis on a certain language, i.e., Java, because it is one of the most popular programming languages in practice[4]. We select 567 Java projects that adopted GA from our dataset. Then, we extract the steps in order from YAML files through the *bashlex* parser library of python. We implement the SPADE (functions *cspade* in R) with support of 0.005. We find that, among all the steps, '*actions/checkout*', '*actions/setup-java*', and '*actions/cache*' are the most frequently used steps. Table V shows the frequent sequences (support$\geq$0.05 and sequence length$>$1). It is noticeable that most of these frequent sequences are begin with '*actions/checkout*', which checks-out repository under *$GITHUB_WORKSPACE*, and in this way, the workflow can access it. Furthermore, '*actions/checkout*' is usually followed by '*actions/setup-java*', '*actions/cache*', and '*actions/upload-artifact*' out of order. The subsequent rules hold true in 47.7%, 30.7%, 15.4%, and so on with respect to the total number of projects. Although, we find 2 rules which occur in more than 30% of the project, the rest of the rules have low existence.

Finding 4 ☛ *Among all the steps,* 'actions/checkout*',* 'actions/setup-java*', and* 'actions/cache*' are the most frequently used steps. We find 2 rules among two steps that present in 47.7% and 30.7% of the total projects, but the occurrences of further rules are rare.*

C. RQ3: Adoption Effects of GA

1) What is the impact of GA on commit frequency?: First, we examine the commit frequency. By following the method proposed by Zhao et al. [34], we distinguish non-merge and merge commits. We analyze merge commits on the main development branch as a proxy for work that would have been subjected to GA. Since not all our projects have merge commits in all 24 periods, we restrict this analysis to only 240 projects that have at least one merge commit in each time window. Fig. 5 gives the boxplots of per-project number of merge commits (top) and non-merge commits (bottom), respectively, for each of 12 consecutive 30-day time intervals before and after the GA adoption. The horizontal line in each boxplot is the median value across all projects.

[4]https://www.tiobe.com/tiobe-index. The TIOBE Programming Community index is an indicator of the popularity of programming languages.

TABLE V
FREQUENT ACTIONS SEQUENCES IN JAVA PROJECTS.

No.	Sequence	Support
1	<actions/checkout, actions/setup-java>	0.477
2	<actions/checkout, actions/cache>	0.307
3	<actions/setup-java, actions/cache>	0.154
4	<actions/checkout, actions/setup-java, actions/cache>	0.148
5	<actions/checkout, actions/upload-artifact>	0.120
6	<actions/cache, actions/setup-java>	0.085
7	<actions/setup-java, actions/upload-artifact>	0.082
8	<actions/checkout, actions/setup-java, actions/upload-artifact>	0.081
9	<actions/checkout, actions/cache, actions/setup-java>	0.081
10	<actions/cache, actions/upload-artifact>	0.069
11	<actions/checkout, actions/cache, actions/upload-artifact>	0.067
12	<actions/cache, actions/cache>	0.063
13	<actions/checkout, actions/cache, actions/cache>	0.056

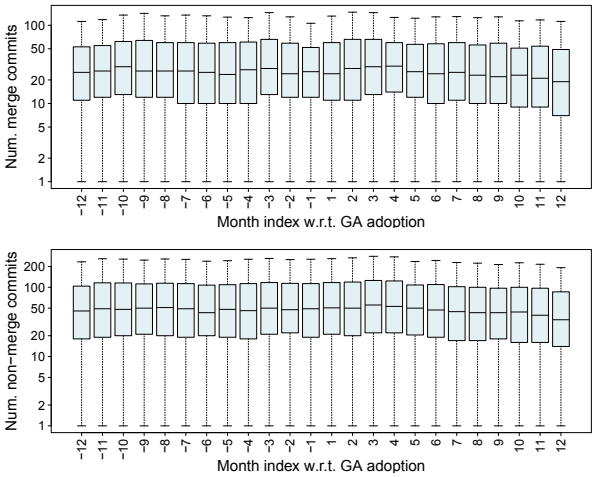

Fig. 5. Commit frequency before and after the GA adoption.

We find that the merge commits appear to display a slightly increasing trend right after GA adoption, and followed by decreasing after that (Fig.5, top). The median number of merge commits per month is 26 before GA adoption and 24 after. We also observe a discontinuity at $t = 0$: the across-projects median is 24 commits right before the adoption period and 22 commits right after the adoption period. For non-merge commits, we observe relative stability in the number of non-merge commits prior to GA adoption (Fig.5, bottom). The non-merge commits appear to display a slight decreasing trend after GA adoption (there are 56 commits in median before adoption, while it drops to 53 commits in the last 12 periods). Besides, there is no discontinuity at $t = 0$.

We fit a mixed-effects RDD model to model trends in the number of merge commits per project over time, while controlling for confounds. Notice that we control for *non-*

TABLE VI
COMMIT FREQUENCY MODEL. THE RESPONSE IS LOG(NUMBER OF MERGE COMMITS) PER MONTH. $R^2_m = 0.50, R^2_c = 0.89$.

	Coeffs (Errors)	Sum sq.
(Intercept)	-0.970 (0.430)*	
log(totalCommits)	0.023 (0.054)	0.04
log(non-mergeCommits)	0.761 (0.010)***	1,218.01***
ageAtGA	-0.001 (0.002)	0.70
log(contributor)	0.107 (0.061)	0.65
time	0.004 (0.003)	0.46
time_after_intervention	-0.009 (0.004)*	1.22*
intervention	0.023 (0.025)	0.18

*** p < 0.001, ** p < 0.01, * p < 0.05

mergeCommits, as these may have not all been subjected to GA, since they appear to display a decreasing trend with time. We model a random intercept for language, to allow for language-to-language variability in the response. Besides, we model a random intervention slope and intercept for projects to allow for project-to-project variability in the response (i.e., projects with lower initial activity may be less strongly affected by adopting GA than high-frequency projects).

Table VI summarizes the results. The fixed-effects part of the model fits the data well (R^2_m=0.50). There is also a considerable amount of variability when adding the random effects (R^2_c=0.89). Among the fixed effects, we observe that the *non-mergeCommits*, our main control, explains most of the variability in the response. The coefficient is positive, i.e., when other conditions remain the same, more merge commits correlate with more non-merge commits. None of *totalCommits*, *ageAtGA* or *contributor* has any statistically significant effects. Next, we turn to our GA variables. The model confirms a statistically significant, negative, baseline trend in the response with *time_after_intervention* (with a small effect). The model does not detect any discontinuity at adoption time, since the coefficient for *intervention* is not statistically significant. Thus, there is a decrease in the slope of the time trend after GA adoption, i.e., fewer merge-commits as time passes. This is inconsistent with the "commit often" guideline of Fowler [6]. One explanation is that projects are switching to more refined and focused development workflows after adopting GA, many commits are migrated to other branches over time, to be further explored in future work.

Finding 5 ☛ *Merge commits frequency decreases slightly over time after switching to GA.*

2) What is the impact of GA on issue resolution?: Next, we examine the issue resolution. Since not all of our projects have recorded issues in all 24 periods, we restrict this analysis to only 1,169 projects that have at least one issue closed in each time window. Fig. 6 shows the boxplots of per-project number of closed issues (top) and mean issue latency (bottom) per unit time period, before and after GA adoption. There is a relative stability in the number of issue closing prior to GA adoption (Fig. 6, top), with around 32 in median, and a short increasing trend after the adoption with 39 issues closed in median in period 1-4. Overall, the number of closed issues appears to display a decreasing trend after the GA adoption.

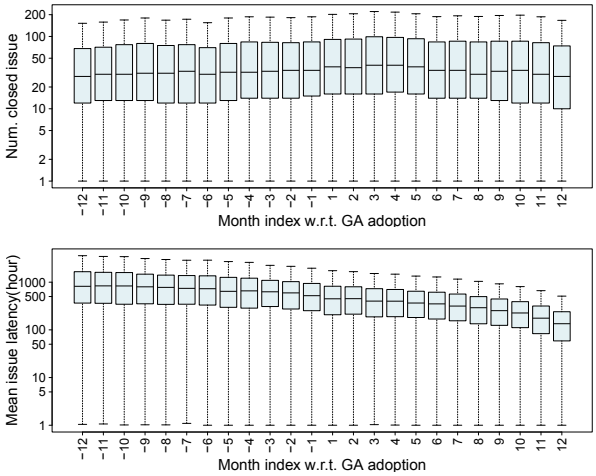

Fig. 6. Issue resolution before and after the GA adoption.

TABLE VII
ISSUE RESOLUTION MODEL. THE RESPONSE IS LOG(NUMBER OF ISSUES CLOSED / MEAN ISSUE LATENCY) PER MONTH.

	Number of issues closed		Mean issue latency	
	Coeffs(Errors)	Sum sq.	Coeffs(Errors)	Sum sq.
(Intercept)	-2.873 (0.164)***		5.598(0.346)***	
log(totalCommits)	0.599 (0.024)***	270.07***	-0.080(0.026)**	10.17**
ageAtGA	-0.014 (0.001)***	117.97***	0.039(0.025)	2.82
log(contributor)	0.436 (0.030)***	93.66***	0.232(0.033)***	55.28***
time	0.015 (0.002)***	37.20***	-0.037(0.002)***	261.10***
time_after_intervention	-0.043 (0.002)***	152.40***	-0.067(0.003)***	415.70***
intervention	0.203 (0.016)***	70.60***	0.028(0.024)	1.55
Marginal R^2	0.450		0.135	
Conditional R^2	0.752		0.476	

*** p < 0.001, ** p < 0.01, * p < 0.05

We also observe a discontinuity at t =0: the across-projects median is 34 issues closed right before the adoption period and 38 issues closed right after the adoption period. Moreover, the median issue latency seems to decrease slightly prior to the introduction of GA (Fig. 6, bottom), and displays a more sharp decline trend afterward.

The statistical models for the number of closed issues and the mean issue latency are summarized in Table VII. As above, the combined fixed-and-random effects models fit the data much better than the basic fixed-effect models, indicating that the project-to-project and language-to-language variability are responsible for most of the variability explained. Regarding closed issues, the results demonstrate that all the variables have statistically significant effects. The coefficient of the *totalCommits* and *contributor* are positive. On the contrary, *ageAtGA* shows a slightly negative effect on the issue closing. There is an increasing time baseline trend pre-adoption and a statistically significant discontinuity at the adoption time; Also, there is a decreasing time baseline trend after-adoption as the sum of the coefficients for *time* and *time_after_intervention*<0. As for the issue latency, we find a consistent decrease in issue resolution latency with time as both of *time* and *time_after_intervention* have a negative coefficient.

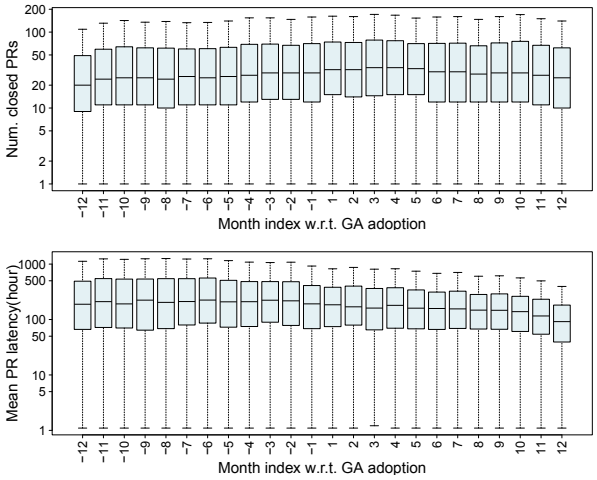

Fig. 7. PR resolution before and after the GA adoption.

Finding 6 ☛ *More issues are being closed over time before GA adoption, but this trend reverses with the switch to GA. Moreover, there is a consistent decrease in issue resolution latency with time, including post-GA adoption.*

3) What is the impact of GA on PR resolution?: The last development practice that we examine is pull request resolution, which we analyze along two dimensions: the number of pull requests closed, and the average pull request latency (i.e., time to close, in hour) per time window. Since not all our projects have PR commits in all 24 periods, we restrict this analysis to only 827 projects that have at least one PR commit in each time window. Fig. 7 shows the boxplots of per-project number of closed PRs (top) and PR latency (bottom) per unit time period. We observe the median number of closed PRs per month is 25 before GA adoption, and 30 after (Fig. 7, top). There is a discontinuity at t =0: the across-projects median is 29 PRs closed right before the adoption period and 32 PRs closed right after the adoption period. We note that the number of PR closed is increasing before the adoption period while decreasing after the adoption period. As for the PR latency (Fig. 7, bottom), we observe relative stability in the latency of PR closing prior to the GA adoption. The median latency seems to decrease after the adoption of GA.

The statistical models for the number of PRs closing and the mean PR latency are summarized in Table VIII. As above, the combined fixed-and-random effects models fit the data much better than the basic fixed-effect models, indicating that the project-to-project and language-to-language variability are responsible for most of the variability explained. Regarding closed PRs, all the variables have statistically significant effects. *TotalCommits* and *contributor* have positive effects on the PR closing. On the contrary, *ageAtGA* has a negative effect. We note an increasing time baseline trend pre-adoption, while the trend turns negative after adopting GA as the sum of the coefficients for *time* and *time_after_intervention*<0; Moreover, there is a statistically significant discontinuity at the adoption time. As for the PR latency, the fewer commits or the more contributors the project has, the longer the latency is. We note a statistically decreasing baseline time trend pre-adoption; no significant discontinuity at the adoption time; and the adoption aggregates the decreasing baseline time trend.

TABLE VIII
PRS RESOLUTION MODEL. THE RESPONSE IS LOG(NUMBER OF PRS CLOSED / MEAN PR LATENCY) PER MONTH.

	Number of PRs closed		Mean PR latency	
	Coeffs(Errors)	Sum sq.	Coeffs(Errors)	Sum sq.
(Intercept)	-3.554(0.233)***		2.896(0.262)***	
log(totalCommits)	0.673(0.032)***	191.07***	-0.101(0.036)**	10.53**
ageAtGA	-0.014(0.001)***	69.98***	0.002(0.001)	2.33
log(contributor)	0.338(0.040)***	30.35***	0.597(0.047)***	218.81***
time	0.023(0.002)***	62.21***	0.003(0.003)	1.57
time_after_intervention	-0.046(0.003)***	123.53	-0.056(0.005)***	201.39***
intervention	0.181(0.019)***	39.62***	-0.030(0.032)	1.20
Marginal R^2	0.397		0.752	
Conditional R^2	0.103		0.451	

*** p < 0.001, ** p < 0.01, * p < 0.05

Finding 7 ☛ *Despite the number of closed PRs increases over time, surprisingly, the initial trend is flattened post-GA adoption, resulting in a downward trend. Also, PRs are taken shorter to close over time after adopting GA.*

V. DISCUSSION

A. Implications

From our qualitative study results, we have distilled some implications for different stakeholders.

1) For researchers: Our studies provide a rich resource of initial ideas for further study. Our statistical results show that nearly 22% of popular projects adopt the GA (**RQ1**), researchers should further investigate the barriers and benefits that developers face when adopting GA, which needs more qualitative analyses, e.g., manual analysis, survey, and interview. Also, we find that the median number of workflows per project is 2.8 (**RQ2**), researchers should further investigate whether certain types of projects will set more workflows in GA than others. Our regression modeling results show that GA adoption has effects on commit frequency, issue and PR's resolution efficiency (**RQ3**). The influence of different GA configuration details (e.g., workflows, jobs, steps) on those project outcomes should be further empirically evaluated. With more data and careful measure classification along different dimensions, some patterns may become apparent. Our findings motivate the need for collecting more empirical evidence that helps developers, who seek to better use GA tool in their projects without arbitrary decisions.

2) For developers: Our study finds that adopting GA is helpful for issue and PR's resolution (**RQ3**). A direct implication is that, if a project has a large burden of tasks, then using GA might be beneficial, as it is a tool that allows developers to organize workflow automatically. Also, our study shows that during GA configuration, developers need to set several components, i.e., workflow, jobs, and steps, which contain a lot of alternatives (**RQ2**). It is challenging for less experienced developers to configure a suitable GA workflow.

Hence, there is a need for a list of "bespoke GA best practices" for developers with different experiences and needs. Although there are diversified steps that can be found from the GitHub Marketplace or created by developers themselves, it is still difficult to build commonly used action sequences, e.g., we only find 2 rules among two steps in those Java projects (**RQ2**). Developers should select appropriate steps. Therefore, the issue of how to manage and help developers configure the best action sequence needs to be addressed.

3) For tool builders: Our study shows that programming languages may correlate with the GA adoption (**RQ1**) and have significant effects on the project outcomes (**RQ3**). However, the current GA configuration process of different language projects mainly depends on developers' personal experience and manual effort. To reduce manual efforts, it is desirable to develop assistive tools to aid GA configuration of projects with specific programming languages. E.g., such tools can help recommend related workflow samples to guide developers toward GA configuration. Also, we find that the projects in our dataset have an average of 1.8 jobs per workflow (**RQ2**), and these jobs run on different runners. The jobs communicate through the *artifacts* which can be created in one job and used in another for deployment in a workflow. However, we further investigate and find that the actions related to *artifacts* have a high cost of running time (mean time of '*actions/download-artifact*' is 37.04s while the mean time of all actions in our dataset is 6.91s). Thus, based on the trade-offs developers face, there are two suggestions: one is providing more guidelines to lower the initiation obstacles for more inexperienced developers. Another is improving the performance of actions to reduce the time cost, e.g., refactoring the '*actions/download-artifact*'.

B. Threat to Validity

We now describe some threats to validity that we have identified in the course of this study.

1) Internal Validity: In the regression modeling, we controlled for the GA age, contributors size, and the total number of commits, and we set the programming language and project name as random-effect variables. But those projects may have many different application domains (e.g., system software, web frameworks, and non-web libraries), which may cause some bias, although in our manual examination, we did not find evidence for it. In future work, we plan to study how the adoption of GA varies in different application domains. Moreover, to reduce internal threats, we applied multiple data filtering steps. We tested the robustness of our models by varying the data filtering criteria (e.g., 9-month windows instead of 12, which consists of more projects), and observed similar phenomena.

2) External Validity: Although our dataset consists of over 27,790 popular projects, the results may not represent other or all GitHub projects. Additionally, we only studied Java projects in the action sequences analysis. We hence leave the analysis of other languages to future work.

VI. Related works

A. GitHub Actions

Because GA is an emerging feature, there have been few studies investigating GA, the only exception is the study by Kinsman et al. [16]. In that paper, the authors investigated the usage and the impact of GA. They found that only a small set (0.7%) of projects adopt GA, and the adoption of GA increases the number of monthly rejected pull requests and decreases the monthly number of commits on merged pull requests. Our work here is substantially different in two aspects:

First, we consider popular projects, a more refined set of studied projects, and investigate configuration details in GA YAML files, enabling powerful quantitative analyses. Second, to explore the impact of GA, they collected 926 GitHub projects for analysis while our dataset consists of 6,246 projects; they considered six months of activity before and after GA adoption while our analysis is based on 12 months. Besides, we investigate different development practices, i.e., commit frequency, issue and PR's resolution efficiency.

B. CI on Open Source Software

Continuous Integration (CI) is a software development practice where developers frequently integrate their work into a common "mainline" repository or branch [18]. More recently, CI is quite popular in Open Source Software (OSS) projects [8], which triggers a series of research studies. Vasilescu et al. [23] found that although most GitHub projects are configured with Travis CI continuous integration service, less than half actually do. They also found that the contribution type and different project characteristics are associated with the success of the automatic builds. Zhao et al. [34] focused on the impact of CI usage on development practices. Gupta et al. [9] analyzed the impact of CI on developer attraction. Cassee et al. [3] focused on the impact of CI on code review. Different from previous studies, our work here aims to understand the usage of GA and provide a more comprehensive investigation of the effects of GA adoption on development practices change.

C. CD and Deployment Pipelines

Continuous deployment (CD) is a software development practice aiming at automating delivery and deployment of a software product, following any changes to its code [11]. Humble et al. [12] reported on an early overview of CD practices and introduced several guidelines. Savor et al. [20] reported on an empirical study conducted in two high-profile Internet companies; they found that the adoption of CD does not limit scalability in terms of productivity in an organization even if the system grows in size and complexity. However, Implementing the automation, i.e., pipelines (workflows), needed to properly provide CD is challenging and takes a lot of time and tuning, due to the many moving parts and the specific needs of each project or organization [11]. The emergence of CD also increases the importance of deployment pipelines [17]. A deployment pipeline should include explicit stages, e.g., building and packaging, to transfer code from a source repository to the production environment [2]. Zhang et

al. [31] investigated the barriers developers face when using containerized CD workflow and the trade-offs developers must make when choosing different CD workflows.

VII. Conclusion

In this paper, we conduct a large-scale empirical study of GitHub Actions (GA): while several GA have been adopted and built by practitioners, relatively little has been done to evaluate the state of practice. We investigate the basic adoption information of GA, i.e., how many popular projects adopt GA in their development process and how presence / absence of the GA correlates with project properties. Then, we analyze the usage details of GA, including its component scale and action sequences. Further, we use RDD to explore the impact of GA adoption on development practices change, i.e., commit frequency, issue resolution, and PR resolution.

Our findings bring to light how practitioners are using, and being impacted by GA, and we are able to distill some implications for different stakeholders.

Acknowledgment

This work was supported in part by the National Key Research and Development Program of China (Grant No.2018AAA0102304), in part by the Foundation of PDL (Grant No.6142110190204), and in part by the Laboratory of Software Engineering for Complex Systems.

References

[1] Jenkins. https://jenkins.io/ Accessed 2021-9-1.

[2] L. Bass, I. Weber, and L. Zhu. *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional, 2015.

[3] N. Cassee, B. Vasilescu, and A. Serebrenik. The Silent Helper: the Impact of Continuous Integration on Code Reviews. In *International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 423–434. IEEE, 2020.

[4] C. Chandrasekara and P. Herath. Artifacts and Caching Dependencies. In *Hands-on GitHub Actions*, pages 51–61. Springer, 2021.

[5] C. Chandrasekara and P. Herath. Introduction to GitHub Actions. In *Hands-on GitHub Actions*, pages 1–8. Springer, 2021.

[6] M. Fowler and M. Foemmel. Continuous Integration, 2006. http://www.martinfowler.com/articles/continuousIntegration.html Accessed 2021-9-1.

[7] E.A. Gehan. A Generalized Wilcoxon Test for Comparing Arbitrarily Singly-Censored Samples. *Biometrika*, 52(1-2):203–224, 1965.

[8] G. Gousios, M. Pinzger, and A. Van Deursen. An Exploratory Study of the Pull-Based Software Development Model. In *International Conference on Software Engineering (ICSE)*, pages 345–355, 2014.

[9] Y. Gupta, Y. Khan, K. Gallaba, and S. McIntosh. The Impact of the Adoption of Continuous Integration on Developer Attraction and Retention. In *International Conference on Mining Software Repositories (MSR)*, page 491–494. IEEE, 2017.

[10] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig. Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects. In *International Conference on Automated Software Engineering (ASE)* pages 426–437. IEEE, 2016.

[11] J. Humble and D. Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Pearson Education, 2010.

[12] J. Humble, C. Read, and D. North. The Deployment Production Line. In *Agile Conference*, pages 6–pp. IEEE, 2006.

[13] R. Hyman. Quasi-experimentation: Design and Analysis Issues for Field Settings. *Journal of Personality Assessment*, 46(1):96–97, 1982.

[14] G.W. Imbens and T. Lemieux. Regression Discontinuity Designs: A Guide to Practice. *Journal of Econometrics*, 142(2):615–635, 2008.

[15] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D.M. German, and D. Damian. An In-Depth Study of the Promises and Perils of Mining GitHub. *Empirical Software Engineering*, 21(5):2035–2071, 2016.

[16] T. Kinsman, M. Wessel, M.A. Gerosa, and C. Treude. How Do Software Developers Use GitHub Actions to Automate Their Workflows? In *International Conference on Mining Software Repositories (MSR)*, pages 420–431. IEEE, 2021.

[17] M. Leppänen, S. Mäkinen, M. Pagels, J. Itkonen, M.V. Mäntylä, and T. Männistö. The Highways and Country Roads to Continuous Deployment. *IEEE Software*, 32(2):64–72, 2015.

[18] M. Meyer. Continuous Integration and its Tools. *IEEE Software*, 31(3):14–16, 2014.

[19] S. Nakagawa and H. Schielzeth. A General and Simple Method for Obtaining R2 from Generalized Linear Mixed-Effects Models. *Methods in Ecology and Evolution*, 4(2):133–142, 2013.

[20] M. Savor, T.and Douglas, M. Gentili, L. Williams, K. Beck, and M. Stumm. Continuous Deployment at Facebook and OANDA. In *International Conference on Software Engineering Companion (ICSE-C)*, page 21–30. ACM, 2016.

[21] M. Shahin, M.A. Babar, and L. Zhu. Continuous Integration, Delivery and Deployment: a Systematic Review on Approaches, Tools, Challenges and Practices. *IEEE Access*, 5:3909–3943, 2017.

[22] D.L. Thistlethwaite and D.T. Campbell. Regression-Discontinuity Analysis: An Alternative to the Expost Facto Experiment. *Journal of Educational Psychology*, 51(6):309, 1960.

[23] B. Vasilescu, S. Van Schuylenburg, J. Wulms, A. Serebrenik, and M.GJ. Van Den Brand. Continuous Integration in a Social-coding World: Empirical Evidence from GitHub. In *International Conference on Software Maintenance and Evolution (ICSME)*, pages 401–405. IEEE, 2014.

[24] C. Vassallo, S. Proksch, A. Jancso, H.C. Gall, and M. Di Penta. Configuration Smells in Continuous Delivery Pipelines: a Linter and a Six-Month Study on GitLab. In *European Software Engineering Conference and Symposium on the Foundations of Software Engineering(ESEC/FSE)*, pages 327–337. ACM, 2020.

[25] M. Wessel, A. Serebrenik, I. Wiese, I. Steinmacher, and M.A. Gerosa. Effects of Adopting Code Review Bots on Pull Requests to OSS Projects. In *International Conference on Software Maintenance and Evolution (ICSME)*, pages 1–11. IEEE, 2020.

[26] M. Wessel, A. Serebrenik, I. Wiese, I. Steinmacher, and M.A. Gerosa. Effects of Adopting Code Review Bots on Pull Requests to OSS Projects. In *International Conference on Software Maintenance and Evolution (ICSME)*, pages 1–11. IEEE, 2020.

[27] Yiwen Wu, Yang Zhang, Tao Wang, and Huaimin Wang. Dockerfile changes in practice: A large-scale empirical study of 4,110 projects on github. In *the 27th Asia-Pacific Software Engineering Conference (APSEC)*, pages 247–256. IEEE, 2020.

[28] Yiwen Wu, Yang Zhang, Tao Wang, and Huaimin Wang. An empirical study of build failures in the docker context. In *the 17th International Conference on Mining Software Repositories*, pages 76–80, 2020.

[29] K. Yamamoto, M. Kondo, K. Nishiura, and O. Mizuno. Which Metrics Should Researchers Use to Collect Repositories: An Empirical Study. In *International Conference on Software Quality, Reliability and Security (QRS)*, pages 458–466. IEEE, 2020.

[30] M.J. Zaki. SPADE: An Efficient Algorithm for Mining Frequent Sequences. *Machine Learning*, 42(1):31–60, 2001.

[31] Y. Zhang, B. Vasilescu, H. Wang, and V. Filkov. One Size Does Not Fit All: An Empirical Study of Containerized Continuous Deployment Workflows. In *European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 295–306. IEEE, 2018.

[32] Yang Zhang, Huaimin Wang, Yiwen Wu, Dongyang Hu, and Tao Wang. Github's milestone tool: A mixed-methods analysis on its use. *Journal of Software: Evolution and Process*, 32(4):e2229, 2020.

[33] Yang Zhang, Gang Yin, Tao Wang, Yue Yu, and Huaimin Wang. An insight into the impact of dockerfile evolutionary trajectories on quality and latency. In *the 42nd Annual Computer Software and Applications Conference (COMPSAC)*, volume 1, pages 138–143. IEEE, 2018.

[34] Y. Zhao, A. Serebrenik, Y. Zhou, V. Filkov, and B. Vasilescu. The Impact of Continuous Integration on Other Software Development Practices: a Large-Scale Empirical Study. In *International Conference on Automated Software Engineering (ASE)*, pages 60–71. IEEE, 2017.